# 2 Linear Regression

## 2.1 Model and Cost Function

**Hypothesis:** $h_\theta(x) = \theta^T x = \sum_{j=0}^{d} \theta_j x_j$

**Cost Function (RSS):**

$$J(\theta) = \frac{1}{2} \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)})^2 = \frac{1}{2} ||X\theta - y||^2$$

### 2.2 Gradient Descent

**Batch Gradient Descent**

$$\theta := \theta - \alpha \nabla_\theta J(\theta)$$
$$\nabla_\theta J(\theta) = X^T(X\theta - y)$$

**Stochastic Gradient Descent**

For each training example $(x^{(i)}, y^{(i)})$:

$$\theta_j := \theta_j - \alpha(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$$

### 2.3 Normal Equations

Setting $\nabla_\theta J(\theta) = 0$:

$$X^T X\theta = X^T y$$

**Closed-Form Solution**

$$\theta = (X^T X)^{-1} X^T y$$

**Note:** Requires $X^T X$ to be invertible.

### 2.4 Probabilistic Interpretation

**Assumption:** $y^{(i)} = \theta^T x^{(i)} + \epsilon^{(i)}$ where $\epsilon^{(i)} \sim \mathcal{N}(0, \sigma^2)$

**Likelihood:**

$$L(\theta) = \prod_{i=1}^{n} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(y^{(i)} - \theta^T x^{(i)})^2}{2\sigma^2}\right)$$

**Maximum Likelihood Estimation leads to minimizing RSS.**

## 3 Logistic Regression

### 3.1 Sigmoid Function

$$g(z) = \frac{1}{1 + e^{-z}}$$

**Properties:**

- $g'(z) = g(z)(1 - g(z))$
- $g(-z) = 1 - g(z)$

### 3.2 Hypothesis and Decision Boundary

**Hypothesis:** $h_\theta(x) = g(\theta^T x) = \frac{1}{1+e^{-\theta^T x}}$

**Decision Rule:** Predict $y = 1$ if $h_\theta(x) \geq 0.5 \Leftrightarrow \theta^T x \geq 0$

### 3.3 Cost Function and Optimization

**Log-likelihood:**

$$\ell(\theta) = \sum_{i=1}^{n} y^{(i)} \log h_\theta(x^{(i)}) + (1 - y^{(i)}) \log(1 - h_\theta(x^{(i)}))$$

**Cost Function:** $J(\theta) = -\ell(\theta)$

**Gradient for Logistic Regression**

$$\frac{\partial J}{\partial \theta_j} = \sum_{i=1}^{n} (h_\theta(x^{(i)}) - y^{(i)}) x_j^{(i)}$$

**Update Rule:** $\theta_j := \theta_j - \alpha(h_\theta(x^{(i)}) - y^{(i)})x_j^{(i)}$

### 3.4 Newton's Method

$$\theta := \theta - H^{-1}\nabla_\theta J(\theta)$$

where $H$ is the Hessian matrix. Also called **Fisher Scoring** for logistic regression.

## 4 Generalized Linear Models (GLMs)

### 4.1 Exponential Family

**Definition 3.** *A distribution is in the* ***exponential family*** *if:*

$$p(y;\eta) = b(y)\exp(\eta^T T(y) - a(\eta))$$

*where:*

- $\eta$*: natural parameter*
- $T(y)$*: sufficient statistic*
- $a(\eta)$*: log partition function*
- $b(y)$*: base measure*

### 4.2 GLM Construction

**Three assumptions:**

1. $y|x;\theta \sim \text{ExponentialFamily}(\eta)$
2. $h_\theta(x) = E[y|x;\theta]$
3. $\eta = \theta^T x$ (natural parameter is linear in $x$)

**Common GLMs**

- **Linear Regression:** $y \sim \mathcal{N}(\mu, \sigma^2)$, $\eta = \mu$, $g(\eta) = \eta$
- **Logistic Regression:** $y \sim \text{Bernoulli}(\phi)$, $\eta = \log\frac{\phi}{1-\phi}$, $g(\eta) = \frac{1}{1+e^{-\eta}}$
- **Poisson Regression:** $y \sim \text{Poisson}(\lambda)$, $\eta = \log\lambda$, $g(\eta) = e^\eta$

# 8 Neural Networks and Deep Learning

## 8.1 Network Architecture

**Multi-layer Perceptron:**

$$z^{[1]} = W^{[1]}x + b^{[1]} \tag{24}$$
$$a^{[1]} = \sigma(z^{[1]}) \tag{25}$$
$$\vdots \tag{26}$$
$$z^{[L]} = W^{[L]}a^{[L-1]} + b^{[L]} \tag{27}$$
$$\hat{y} = \sigma(z^{[L]}) \tag{28}$$

## 8.2 Activation Functions

**Common Activation Functions**

- **Sigmoid:** $\sigma(z) = \frac{1}{1+e^{-z}}$, $\sigma'(z) = \sigma(z)(1-\sigma(z))$
- **Tanh:** $\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$, $\tanh'(z) = 1 - \tanh^2(z)$
- **ReLU:** $\text{ReLU}(z) = \max(0, z)$, $\text{ReLU}'(z) = \mathbf{1}\{z > 0\}$

## 8.3 Backpropagation Algorithm

**Forward Pass:** Compute activations $a^{[l]}$ for $l = 1, \ldots, L$

**Backward Pass:** Compute gradients using chain rule

**Backpropagation Equations**

$$\delta^{[L]} = \nabla_{a^{[L]}} J \odot \sigma'(z^{[L]}) \tag{29}$$
$$\delta^{[l]} = ((W^{[l+1]})^T \delta^{[l+1]}) \odot \sigma'(z^{[l]}) \tag{30}$$
$$\frac{\partial J}{\partial W^{[l]}} = \delta^{[l]}(a^{[l-1]})^T \tag{31}$$
$$\frac{\partial J}{\partial b^{[l]}} = \delta^{[l]} \tag{32}$$

## 8.4 Universal Approximation

Neural networks with one hidden layer can approximate any continuous function on a compact set, given sufficient hidden units.

# 9 Unsupervised Learning

## 9.1 K-Means Clustering

**Algorithm 1** K-Means Algorithm

Initialize cluster centroids $\mu_1, \ldots, \mu_k$ randomly
**repeat**
  **for** $i = 1$ to $n$ **do**
    $c^{(i)} := \arg\min_j ||x^{(i)} - \mu_j||^2$
  **end for**
  **for** $j = 1$ to $k$ **do**
    $\mu_j := \frac{\sum_{i=1}^{n} \mathbf{1}\{c^{(i)}=j\} x^{(i)}}{\sum_{i=1}^{n} \mathbf{1}\{c^{(i)}=j\}}$
  **end for**
**until** convergence

**Objective Function (Distortion):**

$$J(c, \mu) = \sum_{i=1}^{n} ||x^{(i)} - \mu_{c^{(i)}}||^2$$

## 9.2 Principal Component Analysis (PCA)

**Goal:** Find $k$-dimensional subspace that minimizes projection error.

**PCA Algorithm**

1. Center the data: $x^{(i)} := x^{(i)} - \mu$ where $\mu = \frac{1}{n} \sum_{i=1}^{n} x^{(i)}$
2. Compute covariance matrix: $\Sigma = \frac{1}{n} \sum_{i=1}^{n} x^{(i)} (x^{(i)})^T$
3. Compute eigenvectors of $\Sigma$: $\Sigma u_i = \lambda_i u_i$
4. Take top $k$ eigenvectors $u_1, \ldots, u_k$ (principal components)
5. Project data: $y^{(i)} = U_k^T x^{(i)}$ where $U_k = [u_1 \cdots u_k]$

**Optimization Formulation:**

$$\min_{U \in \mathbb{R}^{d \times k}} \sum_{i=1}^{n} ||x^{(i)} - UU^T x^{(i)}||^2 \quad \text{s.t.} \quad U^T U = I$$

**Solution:** $U$ contains the top $k$ eigenvectors of the covariance matrix.

# 10 Bias-Variance Decomposition

## 10.1 Mathematical Decomposition

For a test point $(x, y)$ where $y = h^*(x) + \epsilon$ with $\epsilon \sim \mathcal{N}(0, \sigma^2)$:

**Bias-Variance Decomposition**

$$\text{MSE}(x) = E[(y - \hat{h}(x))^2] \tag{33}$$
$$= \sigma^2 + \text{Bias}^2 + \text{Variance} \tag{34}$$

where:

$$\text{Bias} = E[\hat{h}(x)] - h^*(x) \tag{35}$$
$$\text{Variance} = E[(\hat{h}(x) - E[\hat{h}(x)])^2] \tag{36}$$

### 10.2 Bias-Variance Tradeoff

- **High Bias (Underfitting):** Model too simple, cannot capture true relationship
- **High Variance (Overfitting):** Model too complex, fits noise in training data
- **Optimal Model Complexity:** Balances bias and variance to minimize total error

## 11 Regularization

### 11.1 Ridge Regression (L2 Regularization)

**Cost Function:**

$$J(\theta) = \frac{1}{2}||X\theta - y||^2 + \frac{\lambda}{2}||\theta||^2$$

**Closed-form Solution:**

$$\theta = (X^T X + \lambda I)^{-1} X^T y$$

### 11.2 LASSO (L1 Regularization)

**Cost Function:**

$$J(\theta) = \frac{1}{2}||X\theta - y||^2 + \lambda||\theta||_1$$

**Properties:**

- Promotes sparsity (feature selection)
- No closed-form solution (use coordinate descent or proximal methods)

### 11.3 Elastic Net

$$J(\theta) = \frac{1}{2}||X\theta - y||^2 + \lambda_1||\theta||_1 + \lambda_2||\theta||_2^2$$

## 12 Model Selection and Cross-Validation

### 12.1 Hold-out Validation

Split data into training, validation, and test sets:

- **Training:** Fit model parameters
- **Validation:** Select hyperparameters/model
- **Test:** Final performance evaluation

### 12.2 k-Fold Cross-Validation

**Algorithm 2** k-Fold Cross-Validation

Partition data into $k$ folds
**for** $i = 1$ to $k$ **do**
  Train on folds $\{1, \ldots, k\} \setminus \{i\}$
  Validate on fold $i$
  Record validation error $\epsilon_i$
**end for**
Return average error: $\frac{1}{k} \sum_{i=1}^{k} \epsilon_i$

### 12.3 Leave-One-Out Cross-Validation (LOOCV)

Special case of k-fold CV where $k = n$ (number of training examples).